

Web Application Development

Lecture 11: Express.js Fundamentals

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain what Express.js is and why it is used for web APIs
- Map **HTTP methods** to **CRUD operations** using a REST table
- Recognise common **HTTP status codes** and when to use each
- Access request data via `req.params`, `req.query`, and `req.body`
- Implement all four **CRUD routes**: GET, POST, PUT, DELETE
- Organise routes with `express.Router()` into separate files
- Test API endpoints end-to-end using **Postman**

Agenda

- 1 What is Express.js
- 2 HTTP Methods & REST Table
- 3 req.params / query / body
- 4 GET / POST / PUT / DELETE
- 5 express.Router()
- 6 Postman Testing Workflow
- 7 Recap

Outline

- 1 What is Express.js
- 2 HTTP Methods & REST Table
- 3 req.params / query / body
- 4 GET / POST / PUT / DELETE
- 5 express.Router()
- 6 Postman Testing Workflow
- 7 Recap

What is Express.js?

W3Schools – Express.js

Express.js is a **popular web application framework for Node.js**, offering robust features for building web and API-driven applications. It extends Node.js's core features, providing a **structured approach** to managing server-side logic. Known for its **minimalistic approach**, Express offers essential web application functionalities.

```
1 // 1. Install: npm install express
2 // 2. First Express app
3
4 const express = require('express');
5 const app     = express();
6 const port    = 3000;
7
8 // Define a route for GET /
9 app.get('/', (req, res) => {
10   res.send('Hello World from Express!');
11 });
12
13 // Start the server
14 app.listen(port, () => {
15   console.log(
16     'Server on http://localhost:${port}'
17   );
18 });
```

Terminal

Server on http://localhost:3000

Browser: localhost:3000

Hello World from Express!

Why Express over raw Node.js?

- Much less boilerplate code
- Built-in routing system
- Middleware ecosystem
- JSON parsing built-in
- Used by Netflix, LinkedIn, Uber

Express vs Raw Node.js HTTP

Raw Node.js (verbose):

```
1 const http = require('http');
2
3 const server = http.createServer(
4   (req, res) => {
5     if (req.url === '/' &&
6         req.method === 'GET') {
7       res.writeHead(200, {
8         'Content-Type': 'application/json'
9       });
10      res.end(JSON.stringify({
11        msg: 'Hello'
12      }));
13    } else if (req.url === '/about' &&
14               req.method === 'GET') {
15      res.writeHead(200, {
16        'Content-Type': 'text/plain'
17      });
18      res.end('About page');
19    } else {
20      res.writeHead(404);
21      res.end('Not found');
22    }
23  }
24 );
25 server.listen(3000);
```

Express.js (clean):

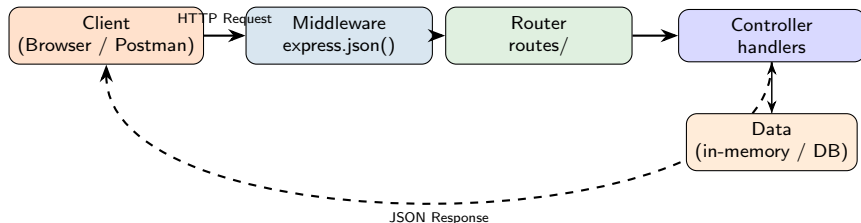
```
1 const express = require('express');
2 const app = express();
3
4 app.get('/', (req, res) => {
5   res.json({ msg: 'Hello' });
6 });
7
8 app.get('/about', (req, res) => {
9   res.send('About page');
10 });
11
12 // Catch-all 404
13 app.all('*', (req, res) => {
14   res.status(404).json({
15     error: 'Not found'
16   });
17 });
18
19 app.listen(3000);
```



Express is the standard

Express handles content-type headers, routing, and status codes automatically – half the lines, far more readable.

Express App Architecture



```
const express = require('express');
const app = express();

app.use(express.json());           // Middleware: parse JSON bodies

const courseRoutes = require('./routes/courses');
app.use('/api/courses', courseRoutes); // Mount router

app.listen(process.env.PORT || 3000);
```

Outline

- 1 What is Express.js
- 2 HTTP Methods & REST Table**
- 3 req.params / query / body
- 4 GET / POST / PUT / DELETE
- 5 express.Router()
- 6 Postman Testing Workflow
- 7 Recap

REST and HTTP Methods

W3Schools – REST API

REST (Representational State Transfer) is an architectural style for building APIs. It maps standard **HTTP methods** to **CRUD operations** on resources. A RESTful API uses URLs to identify resources and HTTP methods to specify what to do with them.

HTTP Method	CRUD	Route example	Description	Body?	Typical status
GET	Read	GET /courses	Retrieve all records	No	200 OK
GET	Read	GET /courses/:id	Retrieve one record	No	200 / 404
POST	Create	POST /courses	Create new record	Yes	201 Created
PUT	Update	PUT /courses/:id	Replace a record	Yes	200 / 404
DELETE	Delete	DELETE /courses/:id	Remove a record	No	200 / 404

💡 RESTful URL conventions

Use **nouns** for resources, never verbs:

- ✓ GET /courses ✗ GET /getCourses
- ✓ POST /courses ✗ POST /createCourse

💡 Plural resource names

Always use **plural nouns**: /courses, /users, /students – not /course, /user, /student.

HTTP Status Codes

Code	Name	When to use
2xx – Success		
200	OK	GET / PUT succeeded
201	Created	POST created a resource
204	No Content	DELETE succeeded (no body)
4xx – Client Error		
400	Bad Request	Invalid data sent
401	Unauthorized	Login required
403	Forbidden	No permission
404	Not Found	Resource does not exist
422	Unprocessable	Validation failed
5xx – Server Error		
500	Internal Error	Unexpected server crash

Express status code usage:

```
1 // 200 OK (default)
2 res.json(data);
3 res.status(200).json(data);
4
5 // 201 Created
6 res.status(201).json(newCourse);
7
8 // 204 No Content
9 res.status(204).send();
10
11 // 400 Bad Request
12 res.status(400).json({
13   error: 'Name is required'
14 });
15
16 // 404 Not Found
17 res.status(404).json({
18   error: 'Course not found'
19 });
20
21 // 500 Server Error
22 res.status(500).json({
23   error: 'Something went wrong'
```

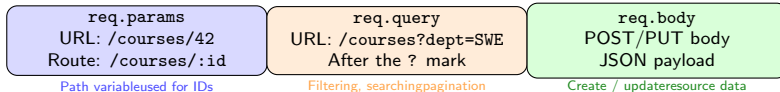
Outline

- 1 What is Express.js
- 2 HTTP Methods & REST Table
- 3 **req.params / query / body**
- 4 GET / POST / PUT / DELETE
- 5 express.Router()
- 6 Postman Testing Workflow
- 7 Recap

Three Ways to Send Data to the Server

W3Schools – Express Request Object

Express gives you three ways to read incoming data: `req.params` for URL path segments, `req.query` for URL query strings, and `req.body` for the request body (JSON).



Property	URL example	Access	HTTP methods
<code>req.params</code>	<code>/courses/42</code>	<code>req.params.id</code>	All
<code>req.query</code>	<code>/courses?dept=SWE</code>	<code>req.query.dept</code>	Mainly GET
<code>req.body</code>	JSON in request body	<code>req.body.title</code>	POST, PUT

req.params – URL Path Parameters

W3Schools – Express Route Parameters

Route parameters are **named URL segments** that capture values at specific positions in the URL. They are specified in the path with a **colon (:) prefix**. Example: `/users/:userId/books/:bookId`

```
1 const express = require('express');
2 const app = express();
3
4 // Single parameter: :id
5 app.get('/courses/:id', (req, res) => {
6   const id = req.params.id;
7   // or: const { id } = req.params;
8   res.json({ courseId: id });
9 });
10
11 // Multiple parameters
12 app.get('/dept/:deptCode/course/:code',
13   (req, res) => {
14     const { deptCode, code } = req.params;
15     res.json({ deptCode, code });
16   }
17 );
18
19 app.listen(3000);
```

GET /courses/42

```
{ "courseId": "42" }
```

GET /dept/SWE/course/381

```
{
  "deptCode": "SWE",
  "code": "381"
}
```

Params are strings

`req.params.id` is always a **string**. Convert with `parseInt()` or `Number()` when needed.

req.query – URL Query Strings

W3Schools – Express Query Parameters

Query parameters are the key-value pairs in the URL **after the ? mark**. You can access them using the `req.query` object. They are mostly used for **searching, sorting, filtering**, and pagination.

```
1 const express = require('express');
2 const app = express();
3
4 // Query params: after the ?
5 app.get('/search', (req, res) => {
6   const { q, category } = req.query;
7   res.send(
8     'Query: ${q}, Category: ${category}'
9   );
10 });
11
12 // Multiple query params
13 app.get('/courses', (req, res) => {
14   const {
15     dept,
16     page = 1,    // default values
17     limit = 10
18   } = req.query;
19
20   res.json({ dept, page, limit });
21 });
```

GET /search?q

Query: express,
Category: js

GET /courses?dept

```
{
  "dept": "SWE",
  "page": "2",
  "limit": "10"
}
```

💡 params vs query

Use **params** for required identifiers (`/courses/42`); **query** for optional filters (`/courses?dept=SWE`).

req.body – Request Body (JSON)

Express.js API Docs

`req.body` contains key-value pairs of data submitted in the **request body**. By default it is `undefined`, and is populated when you use `express.json()` middleware. The body is used with POST and PUT requests to send new or updated data.

```
1 const express = require('express');
2 const app = express();
3
4 // REQUIRED: parse JSON bodies
5 app.use(express.json());
6
7 // POST -- body contains new data
8 app.post('/courses', (req, res) => {
9   const { title, dept, credits } = req.body;
10
11   if (!title) {
12     return res.status(400).json({
13       error: 'Title is required'
14     });
15   }
16
17   const newCourse = {
18     id: Date.now(),
19     title,
20     dept,
21     credits
```

Postman: POST /courses body:

```
{
  "title": "Web App Dev",
  "dept": "SWE",
  "credits": 3
}
```

Response – 201 Created

```
{
  "id": 1710000000000,
  "title": "Web App Dev",
  "dept": "SWE",
  "credits": 3
}
```

 `express.json()` is mandatory

Outline

- 1 What is Express.js
- 2 HTTP Methods & REST Table
- 3 req.params / query / body
- 4 GET / POST / PUT / DELETE**
- 5 express.Router()
- 6 Postman Testing Workflow
- 7 Recap

Building a Courses REST API – Setup

```
1 // index.js
2 require('dotenv').config();
3 const express = require('express');
4 const app = express();
5
6 app.use(express.json()); // parse JSON request bodies
7
8 // In-memory data store (replaces DB for now)
9 let courses = [
10   { id: 1, title: "Web App Dev",      dept: "SWE", credits: 3 },
11   { id: 2, title: "Software Design",  dept: "SWE", credits: 3 },
12   { id: 3, title: "AI Fundamentals",  dept: "CS",   credits: 2 },
13 ];
14 let nextId = 4; // track next ID to assign
15
16 // ---- Routes will go here (next slides) ----
17
18 const PORT = process.env.PORT || 3000;
19 app.listen(PORT, () => {
20   console.log(`SWE381 API running on port ${PORT}`);
21 });
```

```
$ npm run dev
> nodemon index.js
SWE381 API running on port 3000
```

GET – Read All and Read One

```
1 // GET /api/courses
2 // Return ALL courses
3 app.get('/api/courses', (req, res) => {
4   res.status(200).json(courses);
5 });
6
7 // GET /api/courses/:id
8 // Return ONE course by ID
9 app.get('/api/courses/:id', (req, res) => {
10   const id = parseInt(req.params.id);
11
12   const course = courses.find(
13     c => c.id === id
14   );
15
16   if (!course) {
17     return res.status(404).json({
18       error: 'Course not found'
19     });
20   }
21
22   res.status(200).json(course);
23 });
```

GET /api/courses – 200

```
[
  {"id":1,"title":"Web App Dev"},
  {"id":2,"title":"Software Design"},
  {"id":3,"title":"AI Fundamentals"}
]
```

GET /api/courses/2 – 200

```
{
  "id": 2,
  "title": "Software Design",
  "dept": "SWE",
  "credits": 3
}
```

GET /api/courses/99 – 404

```
{"error": "Course not found"}
```

POST – Create a New Resource

```
1 // POST /api/courses
2 // Create a new course
3 app.post('/api/courses', (req, res) => {
4   const { title, dept, credits } = req.body;
5
6   // Validation
7   if (!title || !dept) {
8     return res.status(400).json({
9       error: 'Title and dept are required'
10    });
11  }
12
13  // Build the new course object
14  const newCourse = {
15    id:      nextId++,
16    title:   title.trim(),
17    dept:    dept.trim(),
18    credits: credits || 3
19  };
20
21  // Add to our data store
22  courses.push(newCourse);
23
24  // 201 = Created, return new resource
25  res.status(201).json(newCourse);
26 });
```

Postman: POST /api/courses

```
{
  "title": "Cloud Computing",
  "dept": "CS",
  "credits": 3
}
```

Response – 201 Created

```
{
  "id": 4,
  "title": "Cloud Computing",
  "dept": "CS",
  "credits": 3
}
```

If title missing – 400

```
{
  "error": "Title and dept are
required"
}
```

PUT – Update an Existing Resource

```
1 // PUT /api/courses/:id
2 // Update a course completely
3 app.put('/api/courses/:id', (req, res) => {
4   const id = parseInt(req.params.id);
5   const { title, dept, credits } = req.body;
6
7   // Find the course
8   const index = courses.findIndex(
9     c => c.id === id
10  );
11
12  if (index === -1) {
13    return res.status(404).json({
14      error: 'Course not found'
15    });
16  }
17
18  // Validation
19  if (!title || !dept) {
20    return res.status(400).json({
21      error: 'Title and dept are required'
22    });
23  }
24
25  // Replace with updated data
26  courses[index] = {
```

PUT /api/courses/2 body:

```
{
  "title": "SW Design Patterns",
  "dept": "SWE",
  "credits": 4
}
```

Response – 200 OK

```
{
  "id": 2,
  "title": "SW Design Patterns",
  "dept": "SWE",
  "credits": 4
}
```

💡 PUT vs PATCH

PUT: replace the **entire** resource.

PATCH: update **only provided fields**.

This course uses PUT for simplicity.

DELETE – Remove a Resource

```
1 // DELETE /api/courses/:id
2 // Remove a course
3 app.delete('/api/courses/:id', (req, res) => {
4   const id = parseInt(req.params.id);
5
6   const index = courses.findIndex(
7     c => c.id === id
8   );
9
10  if (index === -1) {
11    return res.status(404).json({
12      error: 'Course not found'
13    });
14  }
15
16  // Remove from array (immutably)
17  const deleted = courses[index];
18  courses = courses.filter(c => c.id !== id);
19
20  // 200 with confirmation
21  res.status(200).json({
22    message: 'Course deleted',
23    deleted
24  });
25
26  // OR: 204 No Content (no body)
```

DELETE /api/courses/3 – 200

```
{
  "message": "Course deleted",
  "deleted": {
    "id": 3,
    "title": "AI Fundamentals",
    "dept": "CS",
    "credits": 2
  }
}
```

DELETE /api/courses/99 – 404

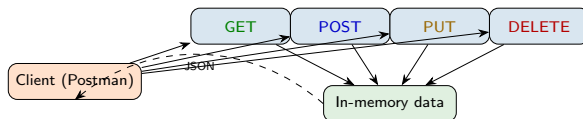
```
{ "error": "Course not found" }
```

Status 200 vs 204

Return **200** with a confirmation body, or **204 No Content** (empty body). Both are acceptable for DELETE.

Complete CRUD API – All Routes at a Glance

Method	URL	Action	Request	Response
GET	/api/courses	Read all	–	200 Array
GET	/api/courses/:id	Read one	params.id	200 / 404
POST	/api/courses	Create	body	201 New obj
PUT	/api/courses/:id	Update	params.id + body	200 / 404
DELETE	/api/courses/:id	Delete	params.id	200 / 404



Outline

- 1 What is Express.js
- 2 HTTP Methods & REST Table
- 3 req.params / query / body
- 4 GET / POST / PUT / DELETE
- 5 **express.Router()**
- 6 Postman Testing Workflow
- 7 Recap

Why Use express.Router()?

Express.js Docs – Router

Use the `express.Router` class to create **modular, mountable route handlers**. A Router instance is a complete middleware and routing system; for this reason, it is often referred to as a “mini-app”.

Without Router (messy index.js):

```
1 // ALL routes crammed into index.js
2 app.get('/api/courses', ...);
3 app.get('/api/courses/:id', ...);
4 app.post('/api/courses', ...);
5 app.put('/api/courses/:id', ...);
6 app.delete('/api/courses/:id', ...);
7
8 app.get('/api/users', ...);
9 app.get('/api/users/:id', ...);
10 app.post('/api/users', ...);
11 // ... grows to hundreds of lines
```

With Router (clean structure):

```
1 // routes/courses.js
2 const router = require('express').Router();
3 router.get('/', getAll);
4 router.get('/:id', getOne);
5 router.post('/', create);
6 router.put('/:id', update);
7 router.delete('/:id', remove);
8 module.exports = router;
9
10 // routes/users.js
11 const router = require('express').Router();
12 // ... user routes
13 module.exports = router;
14
15 // index.js (just 2 lines!)
16 app.use('/api/courses',
17   require('./routes/courses'));
18 app.use('/api/users',
19   require('./routes/users'));
```


express.Router() – Full Implementation

```
1 // routes/courseRoutes.js
2 const express = require('express');
3 const router = express.Router();
4
5 let courses = [
6   { id: 1, title: "Web App Dev",
7     dept: "SWE", credits: 3 },
8   { id: 2, title: "Software Design",
9     dept: "SWE", credits: 3 },
10 ];
11
12 // GET /api/courses
13 router.get('/', (req, res) => {
14   res.status(200).json(courses);
15 });
16
17 // GET /api/courses/:id
18 router.get('/:id', (req, res) => {
19   const course = courses.find(
20     c => c.id === parseInt(req.params.id)
21   );
22   if (!course)
23     return res.status(404)
24       .json({ error: 'Not found' });
25   res.status(200).json(course);
26 });
```

```
1 // POST /api/courses
2 router.post('/', (req, res) => {
3   const { title, dept, credits }
4     = req.body;
5   if (!title)
6     return res.status(400)
7       .json({ error: 'Title required' });
8   const newCourse = {
9     id: Date.now(), title, dept, credits
10  };
11  courses.push(newCourse);
12  res.status(201).json(newCourse);
13 });
14
15 // DELETE /api/courses/:id
16 router.delete('/:id', (req, res) => {
17   const id = parseInt(req.params.id);
18   const exists =
19     courses.some(c => c.id === id);
20   if (!exists)
21     return res.status(404)
22       .json({ error: 'Not found' });
23   courses =
24     courses.filter(c => c.id !== id);
25   res.status(200)
26     .json({ message: 'Deleted' });
```

Mounting the Router in index.js

```
1 // index.js -- clean entry point
2 require('dotenv').config();
3 const express = require('express');
4 const app = express();
5
6 // Middleware
7 app.use(express.json());
8
9 // Mount routers
10 const courseRoutes =
11   require('./routes/courseRoutes');
12 const userRoutes =
13   require('./routes/userRoutes');
14
15 app.use('/api/courses', courseRoutes);
16 app.use('/api/users', userRoutes);
17
18 // Health check
19 app.get('/', (req, res) => {
20   res.json({ status: 'API running' });
21 });
22
23 // 404 for unknown routes
24 app.use((req, res) => {
25   res.status(404).json({
26     error: 'Route not found'
```

Final folder structure:

```
swe381-backend/
  routes/
    courseRoutes.js
    userRoutes.js
  index.js
  package.json
  .env
  .gitignore
```

URL resolution when mounted:

Router path	Full URL
/ (GET)	/api/courses
/:id (GET)	/api/courses/3
/ (POST)	/api/courses
/:id (DELETE)	/api/courses/3



Mount prefix + router path

The router defines paths **relative to the mount point**: /api/courses (mount) +
/:id (router) = /api/courses/:id.

Outline

- 1 What is Express.js
- 2 HTTP Methods & REST Table
- 3 req.params / query / body
- 4 GET / POST / PUT / DELETE
- 5 express.Router()
- 6 Postman Testing Workflow**
- 7 Recap

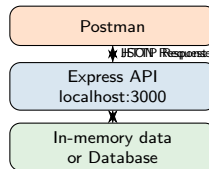
What is Postman?

🚀 Postman – API Testing Tool

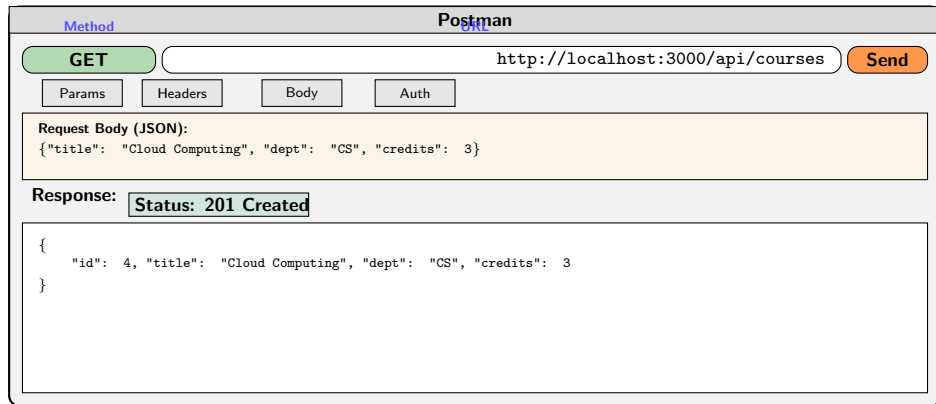
Postman is an **API testing tool** that lets you send HTTP requests to your server and inspect responses – without needing a browser or frontend. It is the standard tool for testing REST APIs during development.

Why use Postman?

- Test API endpoints **before** the frontend is built
- Send POST/PUT requests with **JSON body** easily
- Inspect **response status codes** and headers
- **Save and share** request collections with your team
- Test **authentication tokens** (Bearer JWT)
- Automate test suites



Postman Interface Overview



Postman Testing Workflow – Step by Step

Test 1: GET all courses

```
Method:  GET
URL:     http://localhost:3000/api/courses
Body:    (none)
Expected: 200 OK  [ array ]
```

Test 2: GET one course

```
Method:  GET
URL:     http://localhost:3000/api/courses/1
Body:    (none)
Expected: 200 OK  { object }
         404 if id not found
```

Test 3: POST new course

```
Method:  POST
URL:     http://localhost:3000/api/courses
Headers: Content-Type: application/json
Body:    { "title": "Security",
          "dept": "CS", "credits": 3 }
Expected: 201 Created { new object }
```

Test 4: PUT update course

```
Method:  PUT
URL:     .../api/courses/1
Headers: Content-Type: application/json
Body:    { "title": "Web App Dev v2",
          "dept": "SWE", "credits": 4 }
Expected: 200 OK  { updated object }
```

Test 5: DELETE course

```
Method:  DELETE
URL:     .../api/courses/2
Body:    (none)
Expected: 200 OK  { message: "Deleted" }
         404 if not found
```

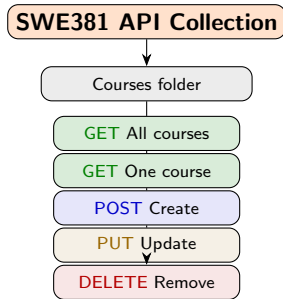


Content-Type header

For POST and PUT, set **Content-Type: application/json** in Postman Headers tab – otherwise Express cannot parse the body.

Postman Collections – Organise Your Tests

A Postman Collection for SWE381 API:



Postman testing best practices:

- Create a **Collection** per project
- Group requests into **folders** by resource
- Set a **base URL variable**: `{{baseUrl}}`
- Test in order: GET first, then POST, PUT, DELETE
- Check both **success** and **error** cases
- Save requests with **descriptive names**



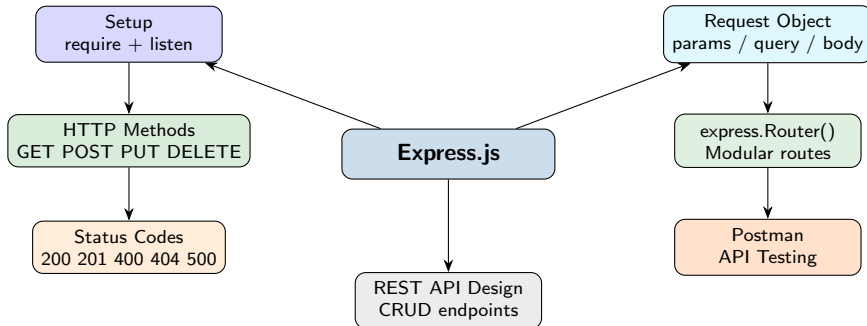
Environment variables

Create a Postman **Environment** with:
`baseUrl = http://localhost:3000`
Then use `{{baseUrl}}/api/courses` in all requests. Switch from local to production by changing one value.

Outline

- 1 What is Express.js
- 2 HTTP Methods & REST Table
- 3 req.params / query / body
- 4 GET / POST / PUT / DELETE
- 5 express.Router()
- 6 Postman Testing Workflow
- 7 Recap**

Express.js – Concept Map



Lecture 11 – Key Takeaways

- 1 **Express.js** is a minimal Node.js web framework that provides structured routing, middleware, and JSON handling with far less code than raw `http`
- 2 REST maps HTTP methods to CRUD: **GET** = Read, **POST** = Create, **PUT** = Update, **DELETE** = Delete
- 3 Use RESTful URL conventions: **plural nouns** (`/courses`), never verbs (`/getCourses`)
- 4 Status codes: **200** OK, **201** Created, **400** Bad Request, **404** Not Found, **500** Server Error
- 5 `req.params` – URL path variables (`/courses/:id`); always a string – convert to number when needed
- 6 `req.query` – query string after `?` (`?dept=SWE`); used for filtering, searching, pagination
- 7 `req.body` – JSON request body (POST/PUT); requires `app.use(express.json())` middleware first
- 8 `express.Router()` creates modular route files – mount with `app.use('/api/courses', router)`
- 9 **Postman** lets you test all CRUD endpoints without a frontend – set Content-Type: `application/json` for POST/PUT

Next Lecture: Mongoose & MongoDB – Connecting the API to a Real Database

Practice Exercises

- 1 Install Express and create a **Hello World** server on port 3000. Add routes for GET / (welcome message) and GET /about (about message). Confirm both work in the browser.
- 2 Create **5 CRUD routes** for a /api/students resource using in-memory data (an array of student objects with id, name, gpa, dept). Implement GET all, GET by id, POST, PUT, and DELETE.
- 3 Add proper **status codes** to every route: 200 for success, 201 for created, 400 if required fields are missing, 404 if the student is not found.
- 4 Add a **search feature** to GET /api/students: support ?dept=SWE as a query parameter that filters results by department.
- 5 Move all student routes into routes/studentRoutes.js using express.Router(). Mount it in index.js at /api/students. Verify all URLs still work.
- 6 Open **Postman** and test all 5 CRUD endpoints. Create a collection with one request per route. Test error cases: GET a non-existent ID, POST without required fields, DELETE a non-existent ID.
- 7 **Challenge:** Add a second resource /api/courses with its own router file. The courses should have title, instructor, and credits. Build a full CRUD implementation and test everything with Postman collections.